sysMonitorData MQTT/TLS/JSON

Interface Design Description

Service ID: *"sysMonitorData"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `sysMonitorData` service of the SysMonitor support system (SysMonitor-Data-MQTT-IDD-5.2.2).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation subscribe . . . 4

4 Data Models **5**
4.1 struct StoredObservation . . . 5
4.2 Primitives . . . 5

5 Go and Rust implementation variants **6**
5.1 Go implementation . . . 6
5.2 Rust implementation . . . 6

6 References **7**

7 Revision History **8**
7.1 Amendments . . . 8
7.2 Quality Assurance . . . 8

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `sysMonitorData` service interface, which provides the stream of all received observations, republished on the local MQTT broker.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	5.0
Data encryption	TLS	1.3 [1], 1.2 [2]
Encoding	JSON	RFC 8259 [3]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitorData` service interface

This document provides the Interface Design Description IDD to the *sysMonitorData – Service Description* document [4].

2.1 Interface registration

```
{
  "serviceDefinitionName": "sysMonitorData",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-Data-MQTT-IDD-5.2.2",
    "schemaVersion": "1.0"
  },
  "interfaces": [
    {
      "templateName": "generic_mqtts",
      "protocol": "ssl",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "mqtt-broker"
        ],
        "accessPort": 8883,
        "baseTopic": "arrowhead/TestCloud/sysmonitor/data",
        "operations": [
          "subscribe"
        ]
      }
    }
```

```
  ]
}
```

Listing 1: Registered service instance (TestCloud).

3 Service Operations

3.1 SUBSCRIBE `<baseTopic>/<observationType>/<providerSystem>`

Operation: subscribe
Output: StoredObservation

A consumer connects to the MQTT broker with mutual TLS (its system certificate) and subscribes below the base topic, e.g. `arrowhead/TestCloud/sysmonitor/data/echo/+` or `arrowhead/TestCloud/sysmonitor/data/+/+`. SysMonitor publishes every stored observation with QoS 1, not retained, with `publishedTime` set; the broker's access list decides who may subscribe.

```
{
  "schemaVersion": "string",
  "observationId": "DataManager-echo-1792670400000-42",
  "providerSystem": "DataManager",
  "serviceInstance": "DataManager",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "receivedTime": "2026-10-22T12:00:00.120Z",
  "publishedTime": "2026-10-22T12:00:00.300Z",
  "value": {
    "status": "alive"
  },
  "quality": "good",
  "pipeline": {
    "routingKey": "TestCloud.sysmonitor.DataManager.DataManager.echo"
  }
}
```

Listing 2: A published observation.

4 Data Models

4.1 struct StoredObservation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	Identifier (publisher's, or a SHA-256 of routing key and content).
"providerSystem"	String	Publishing system.
"serviceInstance"	String	Service instance.
"observationType"	String	Observation type.
"sourceTime"	DateTime	Publisher time, UTC.
"receivedTime"	DateTime	Time SysMonitor received it.
"publishedTime"	DateTime	Time it was forwarded on `sysMonitorData`.
"value"	Object	Observed value.
"unit"	String	Unit.
"quality"	String	Quality.
"pipeline"	Object	Intake details (routing key, exchange).

4.2 Primitives

Type	Description
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-SysMonitor-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8475 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8475"; 8540–8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Authorized AMQPS intake, strict observation validation, atomic dedup/history/outbox, subscriptions, history, MQTT QoS1 forwarding. Java MQTTv5 (Go uses 3.1.1), complete live-broker/UI parity, Java history import, broad schema golden tests. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-SysMonitor-go/documentation/SysMonitor`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-SysMonitor-rust`.

The per-system entry point is declared as a binary of `arrowhead-support-rust` in `ah5-support-runtime-rust/Cargo.toml`. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Listening port. Default 127.0.0.1:8475 (Arrowhead default, role sysmonitor). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or –server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Native mTLS AMQP intake, authorised binding reconciliation, revision subscriptions, SQLite history/outbox, confirmed MQTT output and retention. Full Java DTO/filter parity and wider operational qualification. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/SysMonitor`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[2] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[3] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[4] J. Delsing, “sysMonitorData – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.3.0, 2026, sysMonitor-v5.3.0-sysMonitorData-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)
3	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	